Data Platform System Design

Streaming Clickstream

FastAPI_A → Kafka2 → ClickHouse.raw(kafkaEngine) → Grafana

Zero-copy, high-throughput ingestion of Avro-encoded clickstream events. Reliability and operational safety are ensured via:

- Asynchronous DLQ pattern: Malformed events are redirected to `clickhouse.dead_letter_events`.
- Strict consumer-group isolation: ClickHouse Kafka Engine operates independently from NiFi and archival pipelines, preventing ingestion stalls from cascading.
- Backpressure & Lag Handling:
    - ☐ Kafka acts as the primary buffer. If ClickHouse slows or goes down, lag builds in Kafka.
    - ☐ Lag drain throttling: Controlled via `kafka_max_block_size` and `kafka_poll_max_batch_size` to prevent insert storms on recovery.
    - ☐ Small buffering windows in NiFi absorb minor out-of-order events.
- Late/Out-of-Order Event Handling: Streaming uses `event_ts` for partitioning; `ReplacingMergeTree(FINAL)` ensures late-arriving or duplicate events are deduplicated. Materialized views or scheduled merges avoid heavy `SELECT … FINAL` on hot query paths.
- Schema Governance: Streaming merges enforce schema compatibility via MergeTree constraints; schema evolution is versioned via Schema Registry.

Streaming Operational

FastAPI_B → Microservices(PG + Redis) → Kafka1 → NiFi1 → ClickHouse.enriched → Grafana

- NiFi stops pulling from Kafka if ClickHouse slows, preventing cascading failures.

- `ReplacingMergeTree(event_id)` ensures effectively-once semantics.
- Redis provides hot reference data and rate-limit counters for API.

Batch

FastAPI_C → Postgres → DuckDB(Parquet conversion + file_id validation) → S3 → Airflow → Spark Transform → ClickHouse.master_events → Grafana

- Spark performs joins, denormalization, aggregation.
- Schema enforcement: Spark validates batch data against a formal `master_events` schema contract stored in Postgres. Any new column requires versioned schema updates. Streaming merges enforce compatibility via ClickHouse MergeTree constraints.
- Insert performance: Spark batches writes to ClickHouse in optimal insert sizes (e.g., 10k–100k rows per insert) using native ClickHouse connectors to avoid performance degradation from many small inserts.
- Watermarks: Stored in Postgres; used by Airflow to enable replay-safe incremental batch processing.
- Batch paths can fully rebuild `master_events` from S3 raw if needed.

Archival

Kafka1/Kafka2 → NiFi2 → S3

- Raw, immutable archive of all events.
- Independent consumer groups ensure archival ingestion does not affect streaming or enriched paths.
- Used for disaster recovery and full replay.

---

1. Ingestion Design

Entry Points:

- FastAPI_A → clickstream
- FastAPI_B → internal events
- FastAPI_C → batch CSV uploads

FastAPI Responsibilities:

- Schema validation (Avro + Schema Registry for streaming, CSV schema validation for batch)
- Generates deterministic UUIDv7 `event_id`
- Writes operational state to PostgreSQL
- Reads/writes hot reference data and rate-limit counters via Redis (HA cache)
- Publishes events to Kafka (idempotent producers, `acks=all`)
- Attaches trace_id for observability

Batch CSV:

- Uploaded via FastAPI_C, stored temporarily
- Metadata persisted in PostgreSQL
- Passed to NiFi → S3 Bronze
- NiFi2 consumes from Kafka separately (different consumer group)

S3 Lake ingestion is not dependent on ClickHouse success.

2. Kafka Design

- Cluster: 3–5 brokers, Multi-AZ deployment, Replication Factor = 3, idempotent producers, `acks=all`, `min.insync.replicas=2`

- Topics: Clickstream.events, internal.events, batch.uploads, dlq.*
- Partitioning: clickstream → user_id, internal → aggregate_id, batch → file_id
- Independent consumer groups for ClickHouse and NiFi/archival pipelines
- Retention: clickstream 3–5 days, internal 7–14 days, batch 14–30 days, DLQ 30–90 days, reference topic log-compacted
- Backpressure & Replay: Kafka primary buffer, NiFi pauses on slow ClickHouse, replay via Kafka retention, full rebuild possible from S3 Bronze

Throttling knobs for recovery:

- `kafka_max_block_size` and `kafka_poll_max_batch_size` control how fast ClickHouse Kafka Engine drains backlog after downtime.

3. Storage Design (S3 / MinIO)

- S3 Raw Layer: Immutable, schema-versioned Avro, partitioned by yyyy/mm/dd/hour. System of record, replayable, not analytics-optimized.
- ClickHouse.master_events: Curated, deduplicated, denormalized. Merges streaming + batch, business logic applied, partitioned by `toYYYYMM(event_ts)`, rebuildable from S3 raw.

4. Processing Design

Streaming: FastAPI_A/B → Kafka → ClickHouse (raw/enriched) → Grafana

- At-least-once delivery guaranteed, ordering within partitions
- Deduplication via `ReplacingMergeTree(FINAL)`, idempotent producers

Batch: FastAPI_C → DuckDB → S3 → Airflow → Spark → ClickHouse.master_events

- Schema enforcement, heavy joins, denormalization, aggregation
- Watermarks for replay-safe incremental processing

Data Quality Observability:

- Assertions: row counts, null rates, referential integrity, value distribution checks, schema validation
- Failure Handling: DAG halts, alerts fire, partitions quarantined
- Reconciliation: Scheduled DAG compares ClickHouse counts vs S3 Bronze, triggers alerts/backfill if differences

5. ClickHouse Serving Model

- Sharded + replicated, distributed tables, AZ replication
- Raw_realtime (MergeTree, partition `toYYYYMM(event_ts)`, order `(event_ts, user_id)`)
- Enriched (ReplacingMergeTree(event_id) using FINAL)
- Master_events (pre-joined, batch + streaming)
- TTL policies: Raw 30–90 days, aggregates retained longer
- Serving layer fully rebuildable from S3

6. PostgreSQL (Operational DB)

- Stores API state, upload metadata, job status, reference data, auth, checkpoints, watermarks
- HA: primary + replicas, WAL replication, automated failover

7. Orchestration (Airflow)

- DAGs: Bronze → Spark → Master Events, ClickHouse refresh, DQ, reconciliation, backfill
- Scheduling: Hourly (near-real-time batch), daily (finance aggregates)
- Backfill: parameterized DAGs using watermark logic, reads from Bronze

8. Observability

- Metrics: Kafka lag, under-replicated partitions, leader elections; NiFi queue size/backpressure; ClickHouse insert/query latencies; S3 growth/completeness; Airflow DAG/task metrics
- Logs: centralized (ELK/Loki)
- Tracing: FastAPI → Kafka → ClickHouse trace_id propagation
- Alerts: Grafana alerts on SLA breach, DQ failures

9. Security & Governance

- IAM separation (analytics vs ops)
- Schema Registry enforcement, data contracts
- DQ checks in Spark + Airflow, DLQ for malformed events
- Data lineage tracked via Airflow metadata

10. Disaster Recovery & Reprocessing

- Multi-AZ Kafka, Multi-region S3 replication, ClickHouse replication, Postgres failover, backups
- Short-term replay: Kafka retention

- Long-term rebuild: S3 Bronze

11. Horizontal Scale Points

- FastAPI replicas (load balanced)
- PostgreSQL primary + read replicas, PgBouncer
- Redis Cluster + Sentinel
- Kafka brokers + topic partitions
- NiFi cluster nodes
- Spark executors
- Airflow workers
- ClickHouse shards
- Grafana instances